SCHOOL EOS — MASTER LOW-LEVEL DESIGN

Complete Industry-Standard LLD — Detailed Implementation Baseline

Item	Value
Document	School EOS — Master Low-Level Design **v5.1**
Product boundary	One school / one dedicated product deployment
Backend	NestJS modular enterprise application
Database	PostgreSQL / Supabase PostgreSQL
Authentication	Supabase Auth / configured authentication boundary
API	`/api/v1`; health checks at `/health` and `/ready`
Approved API baseline	**688 endpoints** across Phases 1–11 (per School EOS API Documentation v4.1)
Active login roles	**7** — Admin, Principal, Vice Principal, Faculty, Parent, Finance/Accounts, Hostel Warden
Database authority	PostgreSQL migrations are the executable physical-schema authority

Status	Detailed implementation baseline / LLD freeze candidate
Date	September 2026

Design north star: CORRECTNESS → SECURITY → SAFETY → RELIABILITY → TRACEABILITY → REAL-WORLD USABILITY → SIMPLICITY → EXTENSIBILITY

This edition supersedes v3.0 (and its v4.0 Technical Annex) and v5.0 with a further reconciliation against **School EOS API Documentation v4.1** — the current canonical API source. It preserves every detailed implementation decision from the prior editions rather than compressing them into an executive summary, and adds the role-model and endpoint-count corrections detailed in Section 1.3: **Sports Manager removed** (its structural/asset work absorbed into Admin, its day-to-day operations moved to a new Faculty assignment, **Sports Faculty**), **Academic Coordinator's web access removed** (Faculty is mobile-only, without exception), and the **endpoint baseline corrected from 684 to 688**. New in this v5.1 pass (Section 1.4): the **Physical Training (PT) period is clarified as an ordinary timetabled subject**, governed by the same faculty-subject-assignment mechanism as any other subject — completely independent of, and never substitutable for, the Sports Faculty (team/sport-scoped) assignment.

1. Document Control, Authority & Reconciliation

This LLD is the implementation-level companion to the approved School EOS product baseline, canonical PostgreSQL schema documentation and corrected API documentation. It is not a replacement for PostgreSQL migrations and it does not create a competing schema authority.

Important reconciliation history, in order: the earliest LLD was based on 628 retained APIs; a first correction brought that to 684 endpoints = 628 retained + 56 added across Phases 1–11 (v3.0, and the v4.0 Annex that followed it); **this edition applies a further reconciliation to 688 endpoints against School EOS API Documentation v4.0** — see Section 1.3 for the complete list of what changed in this pass (Sports Manager removed, Sports Faculty introduced, Academic Coordinator web access removed, and an Attendance-phase count correction). This document therefore supersedes every earlier edition wherever endpoint inventory, role model, or product scope conflicts with it. The corrected API source also removes the multi-school/platform model and platform-level Super Admin/Correspondent role.

Authority	Use in implementation
Product design baseline	Business scope, roles, workflows, invariants, design north star
PostgreSQL migrations	Executable physical schema; constraints, indexes, FKs, data types and transaction-visible truth
School EOS API Documentation v4.0	688 endpoint inventory, endpoint paths, access, global API rules, and the current role model (Sports Manager removed, Sports Faculty introduced, Academic Coordinator web access removed)
This LLD	Controllers, DTOs, application services, domain services, policies, repositories, transactions, algorithms, integrations, events, testing and implementation sequencing
ERDs/schema documentation	Logical entity relationships and data architecture; physical migrations remain authoritative

1.1 Source-derived facts retained without silent replacement

- Single-school product; no /schools, /campuses, tenant selector or platform control plane.
- No platform-level Super Admin or Correspondent role.
- **Core login roles (7, current): Admin, Principal, Vice Principal, Faculty, Parent, Finance/Accounts, Hostel Warden.** Transport Manager and Sports Manager/Sports Head are **not** active roles — both were absorbed into Admin (web) for structural/asset ownership, mirroring each other: Transport Manager's vehicles/routes/devices/student-allocation/driver records, and Sports Manager's sport/category definitions, facility records, equipment master data, and coach assignment, are all Admin-only web work now. Neither removal introduced a placeholder role.
- Faculty responsibilities such as Class Advisor, Academic Coordinator, Health In-Charge, stage In-Charge, Community In-Charge, and **Sports Faculty** are assignments/permissions, not separate roles. **None of these assignments — Academic Coordinator included — unlock a web surface; Faculty is mobile-only regardless of assignment.** Sports Faculty is scoped server-side to the specific sport(s)/team(s) the assignment covers and owns sports *operations* (teams, rosters, student sport profiles/participations, trials, training, tournaments, fixtures, results, achievements, certificates, facility bookings, equipment issue/return) — never the structural/asset writes reserved for Admin above.
- Parent access is derived from active student-guardian relationships.
- Student identity is shared across domains; no domain creates a second student master.

- Consequential history is corrected or compensated, not silently overwritten or deleted.
- Financial, wallet, NFC, transport, hostel, health and approval operations use explicit transaction boundaries and idempotency where retry-prone.
- Audit and required outbox records are server-generated and commit atomically with the business state where the event is required.

1.2 Research-informed LLD decisions

The design uses bounded-domain thinking without prematurely splitting the application into microservices. Microsoft guidance recommends a cohesive domain model per bounded context and places persistence concerns behind repositories; that supports a modular monolith here because the product baseline explicitly rejects unjustified distributed infrastructure.

OWASP's API Security Top 10 emphasizes object-level authorization, function-level authorization, sensitive business flows, resource consumption and inventory management. These risks directly reinforce the School EOS requirement that authorization be resolved from role + assignment + scope + object + state rather than frontend visibility.

Supabase Auth issues and refreshes JWTs and keeps its Auth schema outside the auto-generated API. Product tables should therefore map authenticated identities into application-level user accounts, while privileged service credentials remain server-side.

PostgreSQL row locks and transaction isolation are used deliberately for scarce resources and concurrency-sensitive workflows. `FOR UPDATE` can block competing updates until the transaction completes, while serializable/explicit locking strategies must be chosen based on the operation.

1.3 Reconciliation against API v4.0 (688-endpoint baseline; Sports role model corrected)

This edition supersedes the prior v3.0/v4.0-Annex reconciliation (628 → 684 endpoints) with a further reconciliation against **School EOS API Documentation v4.0**, now the single canonical API source (688 endpoints stated; see that document's own note on a pre-existing, carried-over row-count discrepancy, which this LLD does not attempt to silently resolve). The concrete changes this reconciliation makes:

- **Sports Manager is removed as a role**, on top of the earlier Transport Manager removal this LLD had already recorded. Neither leaves a placeholder login. Sports structure/assets (sport & category definitions, facilities, equipment master data, coach assignment) move to Admin, web — the same pattern already used for Transport and Hostel structure. Sports *operations* (teams, rosters, student sport profiles/participations, trials, training sessions, tournaments, fixtures, results, rankings, achievements, certificates, facility bookings, equipment issue/return) move to a new Faculty assignment, **Sports Faculty**, mobile-only, object-scoped to the assigned sport(s)/team(s).
- **Academic Coordinator no longer unlocks a web surface.** It is a Faculty assignment like any other — mobile-only. The heavy screens it previously reached (timetable building, exam setup, bulk report-card generation) remain governed by `Academic authority`, which now resolves only to Admin, Principal, and Vice Principal.
- **Endpoint total corrected to 688** (previously stated as 684). Section 17's reconciliation table is corrected in the same pass: the Attendance phase was undercounted in the earlier baseline (10 endpoints / 1 addition, should be 14 / 5) — a pre-existing error independent of the sports/role changes, fixed here because this LLD's own numbers must match the now-canonical API documentation.
- Every CI/test/manifest reference to the endpoint inventory target (Section 13.3, the Annex's route-inventory and API-implementation-discipline references) is updated from 684 to 688 accordingly, since those are live specifications, not historical narration — the surrounding historical sentences describing the earlier 628→684 correction are left untouched as a record of that milestone.
- **Section 6 was renamed from "56 Newly Added APIs" to "60 Newly Added APIs".** Its Attendance subsection previously detailed only 1 of the 5 endpoints the corrected count requires — the offline-sync endpoint was covered, but the four staff biometric/manual-attendance endpoints (`POST /staff/attendance/biometric-events`, `POST /staff/attendance/manual`, `GET /staff/attendance`, `GET /staff/attendance/{id}`) were missing from the detailed design entirely. They are added here with their access model and key rules, matching the corresponding backend workflow already described in Section 4's Attendance phase.

1.4 Physical Training (PT) period — clarification against API v4.1

School EOS API Documentation v4.1 clarifies that the **PT period is an ordinary timetabled subject, not a Sports Faculty operation.** This LLD reflects that clarification wherever Faculty/Sports Faculty authorization is designed:

- Physical Training is created, assigned, and scheduled through the SAME mechanism as any subject — `SubjectController`/`FacultySubjectAssignmentController`/`TimetableController` (Phase 3 — Classes, Teaching Assignments & Timetable, unchanged in this reconciliation). No new controller, service, or authorization policy was added for it.
- During a PT period, the assigned Faculty member's classroom authority (marking attendance, the offline-sync pattern, substitution mechanics) is resolved by the SAME `v_person_markable_offering`-backed check that

governs every other subject's attendance — see Phase 3 — Attendance below. This check reads the `faculty_subject_assignment` table only; it never consults the Sports Faculty (`SPORTS_FACULTY`) assignment.
- `SportsFacultyScopePolicy` (Phase 4 — Sports, Section 1.3 above) is unaffected and unrelated: it governs sports *operations* (teams, trials, training, tournaments, fixtures, achievements, equipment issue/return), scoped to sport(s)/team(s), and is never consulted for PT-period classroom attendance.
- **The two assignments are independent** and may be held by the same or different Faculty members: a Faculty member can hold the PT `faculty_subject_assignment`, the `SPORTS_FACULTY` assignment, both, or neither. Holding one grants nothing under the other's policy. This is Section 17's new acceptance invariant #31 (below) made explicit at the design level.

2. LLD Architecture Foundation

2.1 Runtime architecture

Client (Web / Mobile / Device) | v API Gateway / NestJS HTTP boundary | +–> Authentication Context | +–> Authorization Context | +–> Controller | v Application Service | +——-+——+

```
        |               |
        v               v
  Domain Services    Integration Adapters
        |               |
        v               v
   Repositories      Providers / Devices
        |
        v
```

PostgreSQL / Supabase | +–> Audit +–> Outbox +–> Idempotency +–> Read Models / Reporting

2.2 Layer responsibilities

Layer	Owns	Must not own
Controller/API	HTTP parsing, DTO validation invocation, auth context extraction, response mapping	Business workflow, direct SQL, transaction orchestration
Application Service	One business operation, orchestration, transaction boundary, calling policies and repositories	HTTP formatting, provider-specific protocol logic
Domain Service	Business invariants, calculations, state transitions, eligibility	HTTP, provider SDKs, persistence details
Authorization Policy	Role + permission + assignment + scope + object + state decisions	UI decisions, credential storage
Repository	Controlled persistence/query operations for a bounded domain	Generic unrestricted CRUD across all tables
Transaction Manager	Begin/commit/rollback and transaction-scoped execution	Business policy
Integration Adapter	Provider/device protocol translation and signature verification	Making external provider data automatically authoritative
Audit Service	Server-generated audit facts	Client-supplied audit truth
Outbox Publisher	Durable post-commit delivery	Changing business state after commit
Read/Reporting Layer	Read-optimized queries and projections	Becoming a second business authority

2.3 Standard request pipeline

REQUEST -> correlation / request ID -> authentication -> account + role resolution -> assignment/scope resolution -> object authorization -> DTO/schema validation -> current-state validation -> domain invariant checks -> transaction -> persistence + audit + outbox + idempotency -> commit -> response

2.4 NestJS module rule

Each bounded module owns its controller(s), application services, domain services, policies, repository interfaces and infrastructure implementations. Cross-domain calls should pass through application contracts rather than reaching directly into another module's repository. modules/ auth/ identity/ access/ admissions/ students/ academics/ attendance/ lms/ assessment/ development/ community/ sports/ finance/ wallet/ canteen/ transport/ nfc/ devices/ hostel/ health/ incidents/ communication/ documents/ approvals/ bulk/ audit/ reporting/ camps/ shared/ auth/ authorization/ validation/ transactions/ idempotency/ outbox/ audit/ storage/ observability/ errors/

3. Shared Cross-Cutting Implementation Design

3.1 Authentication context

Supabase Auth is the authentication authority. The NestJS application validates the access token, resolves the authenticated identity to USER_ACCOUNTS and then loads current product-side roles and assignments. The application must never trust a client-provided role, assignment, student relationship or scope. AuthGuard -> verify access token -> resolve auth subject -> load USER_ACCOUNT -> reject inactive/deactivated account -> attach AuthContext

3.2 Authorization engine

Authorization is a reusable policy engine, not a controller-specific collection of if-statements. authorize(actor, action, resource): authenticate(actor) permissions = resolvePermissions(actor) assignments = resolveAssignments(actor)

```
      scope        = resolveOperationalScope(actor, resource)
      object       = resolveObjectRelationship(actor, resource)
      state        = resolveCurrentState(resource)
      return policyEngine.allow(
          action, permissions, assignments, scope, object, state
      )
```

Dimension	Example
Role	Faculty
Permission	attendance.record
Assignment	Class Advisor / subject teacher
Scope	Class 8-A / Mathematics
Object	Student 123
State	Attendance session OPEN
Action	CREATE / UPDATE / PUBLISH / APPROVE / EXPORT

3.3 Parent object authorization

A parent request containing an arbitrary studentId is never sufficient. The policy must resolve an active student_guardians relationship and permitted relationship type/status before returning or mutating child data. Out-of-scope objects return 404 to avoid object existence disclosure, consistent with the API baseline.

3.4 Validation layers

1. Transport-level validation: JSON shape, content type, size limits and malformed input.
2. DTO validation: types, formats, enum membership, required fields and field-level constraints.
3. Authorization validation: actor permission and object/scope access.
4. State validation: current lifecycle state allows the requested command.
5. Domain validation: business invariants such as date ordering, allocation limits and uniqueness.
6. Persistence validation: PostgreSQL constraints remain the final integrity boundary.

3.5 Error architecture

Code family	Use
VALIDATION_ERROR	Malformed or invalid request
UNAUTHENTICATED	Missing/invalid authentication
FORBIDDEN	Permission/object/function authorization failure
NOT_FOUND	Absent or intentionally hidden object
CONFLICT	State conflict or uniqueness conflict
IDEMPOTENCY_CONFLICT	Same key reused with different request fingerprint
SEMANTIC_RULE_VIOLATION	Domain invariant failure
RATE_LIMITED	Resource consumption protection
DEPENDENCY_UNAVAILABLE	Required provider/database dependency unavailable

3.6 Idempotency service

key = (actorScope, endpoint, idempotencyKey) fingerprint = SHA-256(canonicalRequestBody + relevant route parameters)

BEGIN lock idempotency record if completed: if fingerprint != storedFingerprint -> 409 return stored response create/claim key execute business transaction store outcome COMMIT

Idempotency records must be retained long enough to cover realistic retries and provider/device replay windows. Financial/device commands must bind keys to actor or device scope so a key cannot be replayed by a different principal.

3.7 Audit service

Audit is internal infrastructure. The audit record is created from the server-side action context: actor, action, object, before/after summary where permitted, requestId, timestamp, source/device and reason where required. Clients cannot post arbitrary audit events as if they were authoritative history.

3.8 Outbox

BEGIN mutate business state insert audit event insert outbox event COMMIT

worker: claim outbox rows publish/execute side effect record attempt retry with bounded backoff dead-letter after policy threshold

Downstream notification or integration failure must not roll back an already committed academic, financial or operational action unless the external dependency is explicitly part of the authoritative transaction.

3.9 Optimistic concurrency

Mutable configuration resources use a version column or equivalent If-Match contract where concurrent edits can lose changes. Consequential append-only resources do not use ordinary PATCH as their correction mechanism.

3.10 RLS and server authorization

RLS is defense-in-depth, not a substitute for application authorization. Application policies should be explicit because many business decisions depend on role assignments, state and workflows. Sensitive tables and private storage must have tighter policies.

3.11 Private documents

PostgreSQL stores document metadata, ownership, purpose and business links. Binary content stays in private object storage. Downloads use short-lived controlled access rather than permanent public URLs. Health, finance and evidence documents receive purpose-specific authorization.

4. Domain-by-Domain Low-Level Design

The following sections are the complete implementation design for the approved feature domains. Each domain defines its module boundary, primary aggregates, controllers, application services, policies, repositories, state machines, transaction recipes, concurrency rules and event outputs. The API feature counts are reconciled to the corrected 688-endpoint baseline.

Phase 1 — Authentication, Identity & Session

Approved API allocation: 16 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- AuthController
- SessionController
- MeController
- AdminParentCredentialController

Application services:

- AuthService
- SessionService
- PasswordResetService
- MfaService
- IdentityContextService

Authorization policies:

- AuthPolicy
- SessionOwnershipPolicy
- ParentResetPolicy

Repositories:

- UserAccountRepository
- SessionRepository

- SecurityEventRepository

Domain rules / implementation obligations:

- Authentication -> account mapping -> session -> role resolution
- Parent self-service reset: one successful reset only; failed attempts do not consume allowance.
- Later parent self-service reset is blocked and requires Admin intervention.
- Admin-assisted reset is audited; plaintext credentials are never stored.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 2 — School Academic Configuration & People

Approved API allocation: 31 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- AcademicYearController
- AcademicStageController
- GradeController
- SectionController
- SubjectController
- CalendarController
- PersonController

Application services:

- AcademicConfigurationService
- CalendarService
- PersonService

Authorization policies:

- AdminMasterDataPolicy
- LeadershipReadPolicy

Repositories:

- AcademicYearRepository
- StageRepository
- GradeRepository
- SectionRepository
- SubjectRepository
- CalendarRepository
- PersonRepository

Domain rules / implementation obligations:

- Academic year activation/closure is explicit.
- Historical academic outcomes do not rewrite because current master data changes.

- Reference/master data changes are protected by version checks where necessary.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 2 — User Accounts, Roles & Access

Approved API allocation: 26 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- UserAccountController
- RoleController
- PermissionController
- AssignmentController

Application services:

- AccountProvisioningService
- RoleAssignmentService
- PermissionService
- ResponsibilityAssignmentService

Authorization policies:

- AccountAdminPolicy
- RoleAssignmentPolicy

Repositories:

- UserAccountRepository
- RoleRepository
- PermissionRepository
- UserRoleRepository
- UserAssignmentRepository

Domain rules / implementation obligations:

- Role assignment and faculty responsibility assignments are time-bounded.
- Faculty responsibility does not become a separate login.
- Assignment scope is part of authorization context.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 2 — Students, Guardians & Enrolment

Approved API allocation: 33 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- StudentController
- StaffController
- GuardianController
- StudentGuardianController
- EnrolmentController
- IdentifierController

Application services:

- StudentService
- GuardianService
- StudentGuardianService
- EnrolmentService
- IdentifierService

Authorization policies:

- StudentObjectPolicy
- ParentRelationshipPolicy
- EnrolmentPolicy

Repositories:

- StudentRepository
- GuardianRepository
- StudentGuardianRepository
- EnrolmentRepository
- IdentifierRepository

Domain rules / implementation obligations:

- Student identity is authoritative and shared across domains.
- Guardian relationships are active/effective-dated.
- Ending a relationship preserves history.
- Enrolment transfer/withdrawal is historical, not destructive.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 2 — Admissions

Approved API allocation: 9 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- AdmissionController

Application services:

- AdmissionService

- AdmissionConversionService

Authorization policies:

- AdmissionPolicy
- DuplicateIdentityPolicy

Repositories:

- AdmissionRepository
- StudentRepository
- GuardianRepository
- EnrolmentRepository
- UserAccountRepository

Domain rules / implementation obligations:

- Admission conversion is atomic.
- Duplicate identifiers are checked before creating a new student.
- Parent account provisioning/linking is part of the controlled admission transaction.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 3 — Classes, Teaching Assignments & Timetable

Approved API allocation: 28 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- AcademicClassController
- ClassSubjectController
- TeachingAssignmentController
- TimetableController

Application services:

- ClassService
- TeachingAssignmentService
- TimetableService

Authorization policies:

- AcademicScopePolicy
- TeachingAssignmentPolicy

Repositories:

- AcademicClassRepository
- ClassSubjectRepository
- TeachingAssignmentRepository
- TimetableRepository

Domain rules / implementation obligations:

- Timetable lifecycle: DRAFT -> PUBLISHED -> LOCKED.
- Reject class/period conflicts.
- Reject faculty simultaneous assignments where applicable.
- Published/locked state changes use explicit commands.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 3 — Attendance

Approved API allocation: 10 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- AttendanceSessionController
- AttendanceController
- LeaveController

Application services:

- AttendanceSessionService
- AttendanceRecordingService
- AttendanceCorrectionService
- LeaveService
- OfflineAttendanceSyncService

Authorization policies:

- AttendanceScopePolicy
- AttendanceCorrectionPolicy
- LeavePolicy

Repositories:

- AttendanceSessionRepository
- AttendanceRecordRepository
- AttendanceCorrectionRepository
- LeaveRepository
- IdempotencyRepository

Domain rules / implementation obligations:

- Session lifecycle: OPEN -> IN_PROGRESS -> SUBMITTED -> LOCKED.
- One authoritative record per student/session.
- Locked records require correction workflow.
- Offline sync is independently idempotent per clientKey and returns per-item outcomes.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
```

```
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 3 — Homework, Assignments, LMS & Syllabus

Approved API allocation: 27 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- AssignmentController
- SubmissionController
- LmsMaterialController
- SyllabusController

Application services:

- AssignmentService
- SubmissionService
- LmsMaterialService
- SyllabusService
- ProgressService

Authorization policies:

- AcademicEligibilityPolicy
- SubmissionOwnershipPolicy

Repositories:

- AssignmentRepository
- SubmissionRepository
- LmsMaterialRepository
- SyllabusRepository
- SyllabusProgressRepository

Domain rules / implementation obligations:

- Assignment lifecycle: DRAFT -> PUBLISHED -> SUBMISSION_OPEN -> CLOSED -> ARCHIVED.
- Student eligibility and deadline policy are evaluated server-side.
- Uploaded learning documents remain private.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 3 — Assessment, Examination & Report Cards

Approved API allocation: 38 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- AssessmentTypeController

- AssessmentController
- AssessmentMarkController
- ExaminationController
- GradeScaleController
- ReportCardController

Application services:

- AssessmentService
- MarkService
- MarkCorrectionService
- ExaminationService
- ReportCardService

Authorization policies:

- AssessmentScopePolicy
- MarkCorrectionPolicy
- PublicationPolicy

Repositories:

- AssessmentRepository
- AssessmentMarkRepository
- MarkCorrectionRepository
- ExaminationRepository
- ReportCardRepository

Domain rules / implementation obligations:

- Marks become historical once locked/published.
- Corrections create traceable correction records.
- Report cards are generated from authoritative marks and controlled configuration.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 4 — Student Development & Activities

Approved API allocation: 45 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- ActivityController
- OfferingController
- ParticipationController
- AchievementController
- BehaviourController
- ObservationController

- ProgressController

Application services:

- ActivityService
- ParticipationService
- AchievementService
- DevelopmentRecordService

Authorization policies:

- DevelopmentScopePolicy
- StudentObjectPolicy

Repositories:

- ActivityRepository
- OfferingRepository
- ParticipationRepository
- AchievementRepository
- BehaviourRepository
- ObservationRepository
- ProgressRepository

Domain rules / implementation obligations:

- Development records remain connected to the shared student identity.
- Historical achievements/certificates are not deleted as a convenience.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 4 — Community

Approved API allocation: 13 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- CommunityController
- MembershipController
- CommunityActivityController

Application services:

- CommunityService
- MembershipService
- CommunityActivityService
- CommunityAnnouncementService

Authorization policies:

- CommunityScopePolicy
- CommunityInChargePolicy

Repositories:

- CommunityRepository
- MembershipRepository
- CommunityActivityRepository

Domain rules / implementation obligations:

- Community In-Charge is an assignment on Faculty.
- Exclusive announcements resolve recipients from active membership at publication time.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 4 — Sports

Approved API allocation: 61 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by School EOS API Documentation v4.0. **No Sports Manager role exists.** Every controller below is reached by exactly one of two callers, resolved by the same shared `authorize()` engine (Section 3.2 / Annex C) — never a separate Sports-specific auth path: - **Admin (web)** — structure and assets only: sport/category definitions, facility master records, equipment master inventory, coach assignment. - **Faculty holding the Sports Faculty assignment (mobile/APP)** — operations only: teams, rosters, student sport profiles/participations, trials, training, tournaments, fixtures, results, rankings, achievements, certificates, facility bookings, equipment issue/return — scoped to the sport(s)/team(s) that assignment names, never school-wide.

Controllers:

- SportController *(structure — Admin)*
- SportCategoryController *(structure — Admin)*
- FacilityController *(structure — Admin)*
- EquipmentController *(structure — Admin; master inventory only)*
- CoachAssignmentController *(structure — Admin)*
- ProfileController *(operations — Sports Faculty)*
- TrialController *(operations — Sports Faculty)*
- ParticipationController *(operations — Sports Faculty)*
- TeamController *(operations — Sports Faculty)*
- TrainingController *(operations — Sports Faculty)*
- TournamentController *(operations — Sports Faculty)*
- FixtureController *(operations — Sports Faculty)*
- FacilityBookingController *(operations — Sports Faculty; books against Admin’s facility master)*
- EquipmentIssueController *(operations — Sports Faculty; issues/returns against Admin’s equipment master)*

Application services:

- SportsService *(structure)*
- FacilityService *(structure)*
- EquipmentService *(structure — master inventory CRUD)*
- CoachAssignmentService *(structure — checks `coach.verification_expiry` before assignment, same pattern as a transport driver’s document-expiry check)*

- TrialService *(operations)*
- TeamService *(operations)*
- TrainingService *(operations)*
- TournamentService *(operations)*
- FixtureService *(operations)*
- FacilityBookingService *(operations — reads Admin's facility master, writes only its own booking records)*
- EquipmentIssueService *(operations — reads Admin's equipment master, writes only issue/return records)*

Authorization policies:

- SportsStructurePolicy *(Admin-only; denies every structure/asset write to any Faculty token, Sports Faculty included — the absence of the grant at seed-data level is the enforcement, not a hidden button)*
- SportsFacultyScopePolicy *(resolves ALLOW only when the caller holds the Sports Faculty assignment AND the target team/sport/fixture is inside that assignment's scope; the same Faculty token calling for an unassigned team receives the same DENIAL as any other unauthorized caller)*
- TeamRosterPolicy
- FacilityConflictPolicy

Repositories:

- SportRepository
- FacilityRepository
- EquipmentRepository
- CoachAssignmentRepository
- TeamRepository
- RosterRepository
- TrainingRepository
- TournamentRepository
- FixtureRepository
- FacilityBookingRepository
- EquipmentIssueRepository

Domain rules / implementation obligations:

- Facility bookings protect against overlapping reservations.
- Team rosters are scope-controlled to the assigned Sports Faculty member's sport(s)/team(s).
- Results/rankings/achievements remain historically traceable; achievement writes go through the SAME shared `achievement` repository method the Community module uses (Phase 4 — Community above), never a parallel table.
- A fixture result cannot be timestamped earlier than the fixture's own scheduled start time — enforced in the domain layer, never assumed from client-side date-picker constraints.
- Equipment issue/return operates only against equipment rows that already exist in Admin's master inventory; it cannot create, rename, or retire an equipment record.
- A coach with expired verification cannot be assigned to a team — enforced with a 422 before the write, not a dismissible warning.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
```

```
  }
  return mapped result
```

Phase 5 — Finance, Fees & Payments

Approved API allocation: 43 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- FeeStructureController
- ObligationController
- PaymentController
- RefundController
- ExpenseController
- ReconciliationController
- PaymentEventController
- ExtensionRequestController

Application services:

- FeeService
- ObligationService
- PaymentService
- PaymentAllocationService
- RefundService
- ExpenseService
- ReconciliationService
- FeeExtensionService

Authorization policies:

- FinanceScopePolicy
- PaymentAuthorizationPolicy
- SeparationOfDutiesPolicy

Repositories:

- FeeStructureRepository
- ObligationRepository
- PaymentRepository
- AllocationRepository
- RefundRepository
- ExpenseRepository
- ReconciliationRepository
- ExtensionRequestRepository

Domain rules / implementation obligations:

- Provider-confirmed payment is authoritative.
- Allocation <= confirmed payment.
- Refund <= refundable amount.

- Provider callback uses raw-body HMAC verification and provider event ID replay protection.
- Fee extension request does not change due date until approval.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 5 — Wallet & Canteen

Approved API allocation: 39 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- WalletController
- WalletLimitController
- CanteenController
- CanteenProductController
- MenuController
- PreOrderController
- CanteenTransactionController

Application services:

- WalletService
- WalletLedgerService
- WalletDebitService
- WalletRefundService
- AutoTopUpService
- CanteenSaleService
- PreOrderService
- OfflineSaleSyncService

Authorization policies:

- WalletOwnershipPolicy
- WalletSpendingPolicy
- CanteenScopePolicy
- ParentItemBlockPolicy

Repositories:

- WalletRepository
- WalletLedgerRepository
- WalletLimitRepository
- CanteenRepository
- CanteenProductRepository
- CanteenTransactionRepository
- PreOrderRepository

- AutoTopUpRepository

Domain rules / implementation obligations:

- Wallet balance is derived from authoritative ledger entries.
- Debit is atomic and concurrency-safe.
- Sale amount is server-calculated.
- Sale + wallet debit commit atomically.
- Auto-topup has one active instruction per wallet.
- Pre-order debits at creation; collection is a later operational step.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 6 — Transport, NFC, GPS & Devices

Approved API allocation: 67 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- DeviceController
- VehicleController
- RouteController
- TransportAssignmentController
- TripController
- BoardingController
- NfcCardController
- GpsController

Application services:

- DeviceService
- VehicleService
- RouteService
- TransportAssignmentService
- TripService
- BoardingService
- NfcProvisioningService
- GpsIngestionService
- TransportAlertService

Authorization policies:

- TransportScopePolicy
- DevicePolicy
- CardPolicy
- TripStudentPolicy

Repositories:

- DeviceRepository
- VehicleRepository
- RouteRepository
- TransportAssignmentRepository
- TripRepository
- BoardingRepository
- NfcCardRepository
- TapEventRepository
- GpsTelemetryRepository

Domain rules / implementation obligations:

- Boarding is append-only.
- ALIGHT requires prior BOARD.
- Duplicate/debounce does not create a second event.
- Wrong-bus raises WRONG_BUS.
- Boarding close maps EXPECTED -> NOT_BOARDED and raises safety notification unless approved leave/not-travelling suppresses it.
- Offline events use device/local idempotency.
- NFC card is an identifier, not a wallet.
- GPS telemetry does not create attendance or boarding.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 7 — Hostel

Approved API allocation: 67 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- HostelController
- BlockController
- RoomController
- BedController
- AllocationController
- HostelAttendanceController
- OutingController
- GatePassController
- VisitorController
- HostelIncidentController

Application services:

- HostelService
- AllocationService
- HostelAttendanceService
- OutingService
- GatePassService
- VisitorService
- HostelIncidentService

Authorization policies:

- HostelScopePolicy
- BedAllocationPolicy
- GatePassPolicy

Repositories:

- HostelRepository
- RoomRepository
- BedRepository
- AllocationRepository
- HostelAttendanceRepository
- OutingRepository
- GatePassRepository
- VisitorRepository

Domain rules / implementation obligations:

- Hierarchy: HOSTEL -> BLOCK -> FLOOR -> ROOM -> BED -> ALLOCATION.
- A bed cannot have two incompatible active allocations.
- Allocation is a scarce-resource transaction.
- Gate pass: REQUESTED -> REVIEWED -> APPROVED/REJECTED -> GATE-OUT -> GATE-IN -> CLOSED.
- Hostel attendance is distinct from school attendance.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 8 — Health / Infirmary

Approved API allocation: 43 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- InfirmaryVisitController
- VitalsController
- SickBayController
- MedicationController
- EscalationController

- ReferralController
- FollowUpController
- MedicationScheduleController
- MedicationRoundController

Application services:

- InfirmaryService
- VitalsService
- SickBayService
- MedicationService
- MedicationScheduleService
- MedicationRoundService
- EscalationService
- ReferralService
- FollowUpService

Authorization policies:

- HealthSensitivePolicy
- ParentHealthPolicy
- MedicationConsentPolicy

Repositories:

- InfirmaryRepository
- VitalsRepository
- MedicationRepository
- MedicationScheduleRepository
- MedicationAdministrationRepository
- EscalationRepository
- ReferralRepository
- FollowUpRepository

Domain rules / implementation obligations:

- Health is separately restricted.
- Parent relationship does not grant unrestricted clinical access.
- Standing medication requires active parent consent.
- Overlapping active schedules for the same medicine/student return 409.
- Administration is append-only.
- Overdue doses raise MEDICATION_MISSED.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 9 — Incidents, Discipline, Safety & Emergency

Approved API allocation: 30 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- IncidentController
- DisciplineController
- SafetyController
- EmergencyController

Application services:

- IncidentService
- DisciplineService
- SafetyService
- EmergencyService
- EvidenceLinkService

Authorization policies:

- RestrictedIncidentPolicy
- DisciplinePolicy
- EmergencyPolicy

Repositories:

- IncidentRepository
- DisciplineRepository
- SafetyRepository
- EmergencyRepository
- EvidenceRepository

Domain rules / implementation obligations:

- Historical incidents are not silently deleted.
- Discipline and safety data are restricted.
- Emergency lifecycle is explicit and auditable.
- Evidence is linked to business entities and stored privately.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 9 — Communication, Notifications, Documents & Evidence

Approved API allocation: 10 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- AnnouncementController
- NotificationController

- DeliveryEventController
- DocumentController
- EvidenceController

Application services:

- AnnouncementService
- NotificationService
- DeliveryService
- DocumentService
- EvidenceService

Authorization policies:

- CommunicationScopePolicy
- DocumentPurposePolicy

Repositories:

- AnnouncementRepository
- NotificationRepository
- DeliveryRepository
- DocumentRepository
- EvidenceRepository

Domain rules / implementation obligations:

- Provider delivery callback uses provider authentication/signature.
- Delivery failure never rolls back the originating business transaction.
- Out-of-order delivery events are acknowledged; delivery state is forward-only.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 10 — Approvals

Approved API allocation: 5 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- ApprovalController

Application services:

- ApprovalService
- ApprovalStepService

Authorization policies:

- ApprovalPolicy
- SeparationOfDutiesPolicy

Repositories:

- ApprovalRepository

- ApprovalStepRepository

Domain rules / implementation obligations:

- Requester must not automatically become approver where separation of duties applies.
- Approval state is changed through explicit commands.
- Approval completion can trigger domain-specific state changes.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 10 — Bulk Operations

Approved API allocation: 8 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- BulkJobController

Application services:

- BulkJobService
- BulkValidationService
- BulkExecutionService

Authorization policies:

- BulkScopePolicy
- BulkConfirmationPolicy

Repositories:

- BulkJobRepository
- BulkErrorRepository

Domain rules / implementation obligations:

- Bulk jobs require explicit validation/dry run before confirmation.
- Irreversible processing requires explicit confirmation.
- Row-level failures are reported without losing the job-level audit trail.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 10 — Audit

Approved API allocation: 3 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- AuditController

Application services:

- AuditQueryService
- AuditAnnotationService

Authorization policies:

- AuditReadPolicy

Repositories:

- AuditRepository

Domain rules / implementation obligations:

- Audit events are append-only and server-generated.
- Annotations, if enabled, do not rewrite the original event.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 10 — Reporting & Scheduled Operations

Approved API allocation: 6 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- ScheduledJobController
- ReportingQueryController (internal/read boundary)

Application services:

- ScheduleService
- ReportQueryService
- ReportExportService

Authorization policies:

- ReportScopePolicy
- ScheduleAdminPolicy

Repositories:

- ScheduledJobRepository
- ReadModelRepository

Domain rules / implementation obligations:

- Reports consume authoritative data/read models.
- Exports apply authorization and data-minimization controls.
- Scheduled jobs are explicit, bounded and auditable.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

Phase 11 — Camps

Approved API allocation: 26 endpoint(s). The LLD maps these endpoints to the controllers/services below; exact client-facing method/path contracts remain governed by the corrected API v2.0 source.

Controllers:

- CampController
- CampSessionController
- CampServiceController
- CampParticipantController
- CampConsentController
- CampAttendanceController
- CampCheckupController
- CampFollowUpController
- CampPartnerController

Application services:

- CampService
- CampLifecycleService
- CampParticipantService
- CampConsentService
- CampAttendanceService
- CampCheckupService
- CampFollowUpService
- CampPartnerService

Authorization policies:

- CampScopePolicy
- CampHealthSensitivePolicy
- CampParentConsentPolicy
- CampCompletionPolicy

Repositories:

- CampRepository
- CampSessionRepository
- CampServiceRepository
- CampParticipantRepository
- CampConsentRepository
- CampCheckupRepository
- CampFollowUpRepository
- CampPartnerRepository

Domain rules / implementation obligations:

- Lifecycle: DRAFT -> ANNOUNCED -> ONGOING -> COMPLETED; CANCELLED from any non-terminal state.
- No separate camp login.
- Non-medical camps may have no services.

- Completion is blocked while abnormal findings lack follow-up.
- Checkups freeze after completion.
- Faculty may see attendance but not clinical findings; leadership receives aggregates.
- Parents see only their own child’s permitted results.

Standard command execution:

```
authorize(actor, action, object)
validateCommand(dto)
validateCurrentState(object)
domainService.apply(...)
transaction {
    repository writes
    audit.write(...)
    outbox.enqueue(...)
}
return mapped result
```

5. Critical End-to-End Workflows

5.1 Student onboarding / admission

Admin -> admission created -> duplicate identity validation -> admission submitted/approved -> student created or matched -> guardian

Invariants - Admission conversion is one transaction where the business records must be consistent.

- Unique identifiers protect against duplicate student creation.
- The parent account is an authentication identity plus product USER_ACCOUNT mapping; credentials are not stored in plaintext.
- Optional transport/hostel/community/sports associations are established only through their own domain workflows.

5.2 Academic activity

Faculty resolves assigned class/subject -> opens attendance session -> records attendance -> posts LMS/homework -> assessment -> marks

Invariants - Faculty authorization is assignment-based, not role-only.

- Class Advisor access may be broader than subject teacher access where assigned.
- Locked academic records require controlled correction.

5.3 Finance payment

Parent initiates payment -> provider processes -> provider callback HMAC verified over raw body -> event ID replay checked -> payment

Invariants - Client callback is never authoritative.

- Unknown provider references are acknowledged and queued for reconciliation.
- Payment allocation cannot exceed confirmed payment.
- Refunds compensate prior transactions; they do not delete them.

5.4 Wallet / canteen sale

Terminal resolves student/card -> product price resolved server-side -> wallet debit transaction locks wallet/ledger state -> canteen

Invariants - Client cannot submit authoritative total.

- Frozen wallet fails closed.
- Concurrent debits cannot overspend.
- Historical transaction price is snapshotted.

5.5 Transport boarding

Raw tap -> device authentication -> card resolution -> student resolution -> active transport assignment -> trip

validation -> duplica

Invariants - Raw tap evidence and validated boarding are separate records.

- ALIGHT requires prior BOARD.
- Wrong bus produces WRONG_BUS.
- Boarding close converts EXPECTED to NOT_BOARDED and triggers safety notifications unless suppressed by approved leave/not-travelling.

5.6 Hostel bed allocation

Request allocation -> validate student -> validate hostel/room/bed -> lock candidate bed/active allocation -> verify still free -> cre

Invariants - Two concurrent requests must never both obtain the same bed.

- Use PostgreSQL locking/unique constraints as the final boundary.
- Historical allocations remain queryable.

5.7 Health standing medication

Health staff -> resolve student + consent -> validate schedule overlap/date -> create schedule -> generate expected dose occurrences -

Invariants - No active parent consent means no schedule/no dose.

- Administration is append-only.
- Overdue doses create MEDICATION_MISSED.

5.8 Camp lifecycle

Admin creates camp -> announce -> register participants -> collect consent -> create sessions/services -> record attendance/checkups -

Invariants - Camp has no separate login.

- Health findings are sensitive and are not exposed to ordinary faculty.
- Completion is blocked by unresolved abnormal findings.

6. Detailed Design for the 60 Newly Added APIs

The corrected API source identifies 60 additions: Attendance 5, Finance 3, Wallet/Canteen 11, Transport/NFC 10, Health 4, Communication 1 and Camps 26. The previous LLD omitted these because it was based on the older 628-endpoint baseline; this edition’s own Attendance count is additionally corrected from 1 to 5 (see Section 1.3 and Section 17) — the four endpoints below covering staff biometric/manual attendance were always part of the corrected API source but were never detailed here.

Attendance — 5

Method	Endpoint	Purpose	Access	Key LLD rules
POST	`/api/v1/attendance/sessions/sync`	Offline batch synchronization	Faculty + assigned class scope	Each item carries a client-generated `clientKey` unique per device; `DUPLICATE` is a successful outcome, not an error; items are processed independently (one rejection doesn’t roll back the batch); session lock state is checked at sync time; absence notifications are enqueued after commit and suppressed when approved leave applies.
POST	`/api/v1/staff/attendance/biometric-events`	Record a staff biometric (face/fingerprint) check-in or check-out	Authenticated device (`BIOMETRIC_TERMINAL`)	Device-credential authenticated, not a user token — no person “is” a device; `SIMULATED` / `SIMULATED_FACE` biometric methods are rejected outside non-production environments; a duplicate device idempotency key never creates a second attendance row.

POST	`/api/v1/staff/attendance/manual`	Manual staff attendance entry when the biometric terminal is unavailable	Admin/Principal (direct, confirmed) or Faculty own (self-report, pending approval)	Same endpoint for both callers — the domain service, not the controller, decides the branch: Admin/Principal entry for any staff member writes `state = CONFIRMED` immediately; Faculty entry for their OWN record writes `state = PENDING` and creates an `approval_request` (`STAFF_ATTENDANCE_MANUAL_ENTRY`) in the same transaction. The branch depends on comparing the caller's identity against the `staffId` in the payload — never a client-supplied flag, so a Faculty token can never produce a CONFIRMED result from this endpoint no matter what the request body claims.
GET	`/api/v1/staff/attendance`	List staff attendance events, filterable by staff/date range/method/state	Admin (all) / Faculty (own only)	A `PENDING` staff attendance entry never counts toward attendance percentage or payroll until `CONFIRMED`; Faculty's query is scoped to their own record regardless of filters supplied.
GET	`/api/v1/staff/attendance/{id}`	Get one staff attendance event	Admin / Faculty own / object scoped	Same PENDING/CONFIRMED/REJECTED state model as the list endpoint; out-of-scope objects return 404, not 403.

Controller → application service → repository pattern The endpoint is intentionally thin: authenticate and authorize, validate the command, call the feature application service, execute the domain rules inside the required transaction/idempotency boundary, then return the canonical API envelope. No endpoint writes directly to a table.

Finance — 3

Method	Endpoint	Purpose	Access	Key LLD rules
POST	`/api/v1/finance/obligations/{obligationId}/extension-requests`	Request fee due-date extension	Parent own child / Finance	`requestedDueDate` must be later than the current due date and inside the configured maximum window; allowed obligation states are PENDING, PARTIAL, OVERDUE; creates approval request `FEE_EXTENSION` — it does not itself mutate the due date; one open request per obligation.
GET	`/api/v1/finance/extension-requests`	List extension requests	Finance / parent own	Parent results are restricted to own children.
GET	`/api/v1/finance/extension-requests/{id}`	Get extension request	Object scoped	Object authorization is mandatory; hidden objects return 404.

Controller → application service → repository pattern The endpoint is intentionally thin: authenticate and authorize, validate the command, call the feature application service, execute the domain rules inside the required transaction/idempotency boundary, then return the canonical API envelope. No endpoint writes directly to a table.

Wallet & Canteen — 11

Method	Endpoint	Purpose	Access	Key LLD rules
POST	`/api/v1/wallets/{id}/auto-topup`	Create auto-top-up	Authorized parent	Active provider mandate required; created inactive until mandate is confirmed; one active instruction

				per wallet; `topupAmount > thresholdAmount`; execution becomes a normal wallet ledger credit with an auto-topup source.
GET	`/api/v1/wallets/{id}/auto-topup`	Get active auto-top-up	Object scoped	Only the permitted wallet owner/role may read it.
PATCH	`/api/v1/wallet-auto-topups/{id}`	Update threshold/amount	Authorized parent	Use optimistic versioning; preserve audit trail.
POST	`/api/v1/wallet-auto-topups/{id}/cancel`	Cancel instruction	Parent / Finance	Cancellation does not revoke the underlying bank mandate.
GET	`/api/v1/canteen/menu-items`	List menu items	Vendor / Finance / parent	Allergen tags are controlled vocabulary.
POST	`/api/v1/canteen/menu-items`	Create/update menu item	Vendor / Admin	Price changes do not alter historical transactions.
POST	`/api/v1/canteen/pre-orders`	Create and pay for pre-order	Authorized parent	Wallet is debited at creation; one pre-order per student/vendor/date; insufficient balance returns 422.
GET	`/api/v1/canteen/pre-orders`	List pre-orders	Vendor / parent own	Object and student relationship scope required.
POST	`/api/v1/canteen/pre-orders/{id}/collect`	Collect pre-order	Authorized terminal	Valid only on `forDate`; tap reference recorded; idempotent.
POST	`/api/v1/canteen/pre-orders/{id}/cancel`	Cancel/refund pre-order	Parent / Vendor	Only uncollected eligible orders; refund is a compensating ledger entry.
POST	`/api/v1/canteen/transactions/sync`	Offline canteen sale sync	Authenticated device	Per-item idempotency; sale amount recalculated server-side; wallet debit fails closed if the blocklist is stale/unsafe.

Controller → application service → repository pattern The endpoint is intentionally thin: authenticate and authorize, validate the command, call the feature application service, execute the domain rules inside the required transaction/idempotency boundary, then return the canonical API envelope. No endpoint writes directly to a table.

Transport/NFC — 10

Method	Endpoint	Purpose	Access	Key LLD rules
POST	`/api/v1/transport/trips/{tripId}/boardings`	Record boarding/alighting event	Transport operator/device	Validate device, card, student, trip, assignment and timestamp; append-only; `ALIGHT` requires a prior `BOARD`.
GET	`/api/v1/transport/trips/{tripId}/boardings`	List trip boardings	Transport scoped	Read only after trip/object authorization.
GET	`/api/v1/transport/trips/{tripId}/student-status`	Per-student trip status	Transport / leadership	Status derived from expected assignment + authoritative boarding events.
GET	`/api/v1/transport/trips/{tripId}/exceptions`	List not-yet-boarded students	Transport / attendant	Exception list is operationally derived; approved leave/not-travelling can

				suppress alerts.
POST	`/api/v1/transport/trips/{tripId}/boarding/close`	Close boarding	Transport operator/device	`EXPECTED -> NOT_BOARDED`; repeated close is idempotent; alerts parents except suppressed cases.
GET	`/api/v1/students/{studentId}/transport/status`	Current student transport status	Parent own child / Transport	Parent relationship required.
GET	`/api/v1/students/{studentId}/transport/live`	Current active vehicle position	Parent own child / Transport	Current active trip only; no historical vehicle-position access through this endpoint.
POST	`/api/v1/nfc/cards/{id}/provisioning-token`	Issue provisioning token	Admin + card scope	Single use, short expiry, card-bound; never returns cryptographic key material.
POST	`/api/v1/nfc/cards/{id}/provisioning/confirm`	Confirm provisioning	Personalisation station	Consumes token and transitions card to ACTIVE; audited.
GET	`/api/v1/nfc/cards/blocklist`	Read blocklist for terminal sync	Device credential	Versioned incremental/full snapshot; a stale blocklist refuses payment taps but can continue accepting attendance taps.

Controller → application service → repository pattern The endpoint is intentionally thin: authenticate and authorize, validate the command, call the feature application service, execute the domain rules inside the required transaction/idempotency boundary, then return the canonical API envelope. No endpoint writes directly to a table.

Health — 4

Method	Endpoint	Purpose	Access	Key LLD rules
POST	`/api/v1/health/students/{studentId}/medication-schedules`	Create standing medication schedule	Authorized health staff	Active parent consent mandatory; overlapping active medicine schedules return 409; dates validated; expected doses generated.
GET	`/api/v1/health/students/{studentId}/medication-schedules`	List standing schedules	Health scope / parent own	Parent sees own child only.
POST	`/api/v1/health/medication-schedules/{id}/stop`	Stop schedule	Health staff	Reason required; historical administration unaffected.
GET	`/api/v1/health/medication-rounds`	List due medication doses	Health staff / Warden	Ordered by scheduled time then hostel room; overdue unadministered dose raises `MEDICATION_MISSED`; administration is append-only.

Controller → application service → repository pattern The endpoint is intentionally thin: authenticate and authorize, validate the command, call the feature application service, execute the domain rules inside the required transaction/idempotency boundary, then return the canonical API envelope. No endpoint writes directly to a table.

Communication — 1

Method	Endpoint	Purpose	Access	Key LLD rules
POST	`/api/v1/notifications/delivery-`	Provider delivery	Verified	No Bearer token;

	events	callback	provider/internal	provider signature authentication; provider event ID is the idempotency key; forward-only delivery state; a delivery failure never rolls back the originating business transaction.

Controller → application service → repository pattern The endpoint is intentionally thin: authenticate and authorize, validate the command, call the feature application service, execute the domain rules inside the required transaction/idempotency boundary, then return the canonical API envelope. No endpoint writes directly to a table.

Camps — 26

Method	Endpoint	Purpose	Access	Key LLD rules
GET	/api/v1/camps	List camps	Scoped	Scope to target audience/operational assignment.
POST	/api/v1/camps	Create camp	Admin	Validate dates and target scope; no separate camp login.
GET	/api/v1/camps/{id}	Get camp	Scoped	Object policy required.
PATCH	/api/v1/camps/{id}	Update camp before start	Admin	Mutable only while allowed by lifecycle.
POST	/api/v1/camps/{id}/announce	Announce camp	Admin / Principal	Transition DRAFT -> ANNOUNCED; publish recipient event.
POST	/api/v1/camps/{id}/start	Start camp	Admin / organiser	Transition ANNOUNCED -> ONGOING.
POST	/api/v1/camps/{id}/complete	Complete camp	Admin / organiser	Blocked by abnormal findings without follow-up; freezes checkups.
POST	/api/v1/camps/{id}/cancel	Cancel camp	Admin / Principal	Allowed from any non-terminal state.
GET	/api/v1/camps/{id}/sessions	List sessions	Scoped	Session belongs to camp.
POST	/api/v1/camps/{id}/sessions	Create session	Admin / organiser	Validate date/time within camp bounds.
GET	/api/v1/camps/{id}/services	List services	Scoped	Non-medical camps may have no services.
POST	/api/v1/camps/{id}/services	Define service	Admin / health authority	Service defines type and checkup structure.
GET	/api/v1/camps/{id}/participants	List participants	Organiser / faculty scoped	Scope to permitted participant population.
POST	/api/v1/camps/{id}/participants	Register participants	Admin / organiser	Validate student eligibility and consent requirements.
POST	/api/v1/camps/{id}/consents	Record parent consent	Authorized parent	Active student-guardian relationship required.
POST	/api/v1/camp-participants/{id}/attendance	Record attendance	Faculty / organiser	Attendance belongs to camp session and participant.
GET	/api/v1/camp-participants/{id}/checkups	List checkups	Restricted health / parent own	Clinical findings are sensitive.
POST	/api/v1/camp-participants/{id}/checkups	Record checkup	Health staff	One checkup per participant/service; abnormal findings require follow-up.
GET	/api/v1/camp-checkups/{id}	Get checkup	Restricted health / parent own	Object and purpose scope.
PATCH	/api/v1/camp-checkups/{id}	Correct checkup before completion	Health authority	After completion requires controlled override/reason.

GET	`/api/v1/camps/{id}/follow-ups`	List follow-ups	Health scope / organiser permitted	Follow-ups are linked to findings.
POST	`/api/v1/camp-checkups/{id}/follow-ups`	Create follow-up	Health authority	Action, due date and assignee required.
POST	`/api/v1/camp-follow-ups/{id}/complete`	Complete follow-up	Assignee / health authority	Completion recorded and parent notified where applicable.
GET	`/api/v1/camp-partners`	List camp partners	Scoped	Partner metadata is non-clinical.
POST	`/api/v1/camp-partners`	Create camp partner	Admin	Validate partner identity and active status.
GET	`/api/v1/students/{studentId}/camps`	Student camp history/results	Parent own child / authorized staff	Parent sees permitted results; body measurements only to parent/health staff; no student-facing comparative disclosure.

Controller → application service → repository pattern The endpoint is intentionally thin: authenticate and authorize, validate the command, call the feature application service, execute the domain rules inside the required transaction/idempotency boundary, then return the canonical API envelope. No endpoint writes directly to a table.

7. State Machines & Lifecycle Commands

Resource	Lifecycle	Rules
Academic year	DRAFT -> ACTIVE -> CLOSED	• Activation and closure are explicit commands.

- Closed year is not mutated through normal PATCH.

Timetable	DRAFT -> PUBLISHED -> LOCKED	• Published/locked timetable changes use explicit commands.
Attendance session	OPEN -> IN_PROGRESS -> SUBMITTED -> LOCKED	• Locked attendance requires correction workflow.
Assignment	DRAFT -> PUBLISHED -> SUBMISSION_OPEN -> CLOSED -> ARCHIVED	• Submission is state-gated.
Assessment	DRAFT -> LOCKED -> PUBLISHED	• Locked marks require correction records.
Admission	DRAFT -> SUBMITTED -> APPROVED/REJECTED -> CONVERTED/WITHDRAWN	• Conversion is transactional.
Gate pass	REQUESTED -> REVIEWED -> APPROVED/REJECTED -> GATE-OUT -> GATE-IN -> CLOSED	• Movement history is append-only.
Camp	DRAFT -> ANNOUNCED -> ONGOING -> COMPLETED; CANCELLED from non-terminal	• Completion requires follow-up closure for abnormal findings.
Approval	PENDING -> APPROVED/REJECTED	• Requester cannot approve where SoD applies.
Emergency	DECLARED -> RESPONSE -> RESOLVED -> CLOSED	• Transitions are explicit and audited.

8. Transaction, Concurrency & Consistency Recipes

8.1 Admission transaction

1. Validate duplicate identifiers.
2. Create/update student.
3. Create guardian relationship.
4. Create enrolment.
5. Provision/link parent account.
6. Insert audit/outbox.
7. Commit.

Concurrency: SERIALIZABLE where necessary or explicit uniqueness/locking depending on migration design.

8.2 Payment confirmation

1. Verify provider signature.
2. Claim provider event ID idempotency.

3. Validate event ordering.
4. Create/advance payment state.
5. Audit/outbox.
6. Commit.

Concurrency: Provider callback is provider-authenticated; no user Bearer token.

8.3 Payment allocation

1. Lock payment/remaining amount.
2. Lock target obligation(s) in deterministic order.
3. Validate total allocation <= confirmed payment.
4. Create allocations.
5. Audit.
6. Commit.

Concurrency: Deterministic lock ordering reduces deadlock risk.

8.4 Wallet debit

1. Lock wallet row.
2. Read authoritative ledger/balance state.
3. Validate wallet ACTIVE and sufficient funds/limits.
4. Insert debit ledger entry.
5. Insert business sale if coupled.
6. Audit/outbox.
7. Commit.

Concurrency: Use row-level locking; do not rely on a client-provided balance. PostgreSQL row locks block conflicting writers until transaction end.

8.5 Hostel bed allocation

1. Lock candidate bed/active allocation.
2. Recheck availability.
3. Create allocation.
4. Audit.
5. Commit.

Concurrency: Database unique/exclusion constraints plus row locking are the final safety boundary.

8.6 Boarding close

1. Lock trip.
2. Load expected active assignments.
3. Load authoritative boardings.
4. Derive missing students.
5. Insert NOT_BOARDED facts or state records according to schema.
6. Queue safety notifications.
7. Audit.
8. Commit.

Concurrency: Repeated close returns the prior outcome; it must not create duplicate alerts/events.

8.7 Medication schedule

1. Verify consent.
2. Lock active schedule set for student/medicine.
3. Check overlap.
4. Create schedule/dose occurrences.
5. Audit/outbox.
6. Commit.

Concurrency: Unique/partial indexes should protect the no-overlap invariant where representable.

8.8 Camp completion

1. Lock camp.
2. Check state ONGOING.
3. Query abnormal findings lacking follow-up.
4. Reject if count > 0.
5. Freeze checkups.
6. Mark COMPLETED.
7. Audit/outbox.
8. Commit.

Concurrency: Completion is a business gate, not a frontend button.

8.9 Fail-open vs fail-closed

Operation	Mode	Reason
Payment confirmation	FAIL CLOSED	Financial correctness
Wallet debit	FAIL CLOSED	Cannot overspend
Access grant / authoritative card payment	FAIL CLOSED	Safety/security
Attendance sync	FAIL OPEN + reconcile	Avoid losing attendance during connectivity failure
Transport boarding	FAIL OPEN + reconcile	Do not strand a child because synchronization is temporarily unavailable

The corrected API baseline explicitly adds this rule for stale device state: payment/wallet/access fail closed; attendance/boarding fail open and reconcile.

9. Device, Offline & Integration Architecture

9.1 Offline queue

DEVICE ACTION -> local validation -> durable local event record -> local queue -> retry with backoff -> server sync -> server idempotency check -> authoritative validation -> transaction -> per-item ACK -> local completion

Offline device protocols must distinguish evidence from validated business events. A device may provide raw evidence, but the server decides whether that evidence creates a boarding, sale or other authoritative business fact.

9.2 GPS ingestion

GPS telemetry is high-volume operational evidence. It should have a dedicated ingestion path/worker and not share latency-sensitive interactive controller code. Telemetry does not create attendance or boarding by itself.

9.3 Payment provider callback

raw HTTP body -> preserve bytes -> verify HMAC signature -> parse provider payload -> claim provider event ID -> forward-only state machine -> commit -> 200 acknowledgement

The corrected API source requires HMAC verification over the raw request body before parsing, provider event ID

replay protection, no normal Bearer authentication, forward-only state transitions and reconciliation for unknown payment references.

9.4 Notification provider callback

Notification delivery callbacks use the same adapter boundary: verify provider authenticity, translate provider state to an internal delivery state, enforce forward-only transitions, store provider event ID for replay protection, and never mutate the originating business transaction.

10. Data Access & Persistence LLD

10.1 Persistence rule

The schema documentation defines PostgreSQL migrations as the executable physical authority. The LLD therefore treats repositories as adapters to the approved physical schema rather than as permission to invent tables.

10.2 Repository interface rule

interface WalletRepository { getForUpdate(walletId, tx): Wallet getLedgerBalance(walletId, tx): Money saveLedgerEntry(entry, tx): LedgerEntry }

interface HostelBedRepository { getForUpdate(bedId, tx): Bed getActiveAllocation(bedId, tx): Allocation | null createAllocation(allocation, tx): Allocation }

10.3 Query discipline

- Queries must select only fields required by the use case, especially for health and finance.
- List APIs must be paginated and indexed by actual access patterns.
- Object-scoped queries should encode authorization-relevant predicates where practical as defense-in-depth.
- No repository may silently bypass the application authorization policy.
- Historical reads must resolve effective-dated relationships correctly.
- Read models may denormalize for dashboards but never become authoritative for commands.

10.4 Historical data

Data	Mutation strategy
Published marks	Correction record / compensating change
Locked attendance	Correction workflow
Payment	Compensating refund/reversal
Wallet ledger	Compensating entry
Tap event	Append-only
Boarding	Append-only
Medication administration	Append-only
Gate movement	Historical operational event
Audit	Append-only
Telemetry	Append-only / retention policy

11. Observability, Reliability & Background Processing

11.1 Correlation

Every request receives a requestId. The identifier propagates to application logs, audit records, outbox entries and provider calls where possible.

11.2 Structured logging

Field	Rule
requestId	Always present
actorId	Present for user actions; omitted/redacted for anonymous probes
deviceId	Present for device actions
endpoint	Route template, not sensitive payload
durationMs	Request duration
result	Success/failure code

domainObjectId	Only where safe and useful
errorCode	Canonical application error

11.3 Metrics

- HTTP latency/error rate by route template.
- Authorization denials by policy family.
- Idempotency conflicts and duplicate replays.
- Payment callback verification failures and unknown-reference reconciliation queue.
- Wallet debit contention and failed insufficient-funds attempts.
- Transport boarding acceptance/rejection and offline backlog.
- GPS ingestion lag and stale device counts.
- Hostel allocation conflicts.
- Medication overdue counts and MEDICATION_MISSED alerts.
- Camp follow-up overdue counts.
- Outbox backlog, retry count and dead-letter count.

11.4 Background workers

Worker	Responsibility
Outbox worker	Publish committed domain events / notifications
Payment reconciliation	Resolve unknown provider references and discrepancies
GPS ingestion	Normalize/store telemetry and health signals
Notification worker	Send notifications and process retries
Medication round evaluator	Mark overdue doses and raise alerts
Camp follow-up evaluator	Raise FOLLOWUP_OVERDUE alerts
Bulk worker	Validate/execute confirmed bulk jobs
Scheduled job worker	Execute explicitly configured schedules

12. Security, Privacy & Safety LLD

12.1 Threat-driven controls

Threat	Control
Broken object authorization	Relationship/object policies; 404 for hidden objects
Broken function authorization	Permission + assignment checks on every command
Property over-posting	DTO allowlists; no direct entity binding
Sensitive business flow abuse	Rate limits, idempotency, state machines, explicit confirmation
Credential compromise	Supabase Auth, token validation, session revocation, MFA where policy enables
Provider callback spoofing	Raw-body HMAC/signature verification
Replay	Idempotency key/provider event ID/device local event ID
Data leakage	Field minimization and private storage
SSRF	Allowlisted provider endpoints; no arbitrary URL fetch from client input
Inventory drift	API inventory test against approved 688-endpoint baseline

These controls directly address the authorization, authentication, property-level authorization, sensitive business-flow, SSRF, misconfiguration and inventory concerns highlighted by OWASP’s API Security Top 10.

12.2 Health privacy

- Health is a separately restricted domain.
- Parent access is purpose/scope controlled; relationship alone is not unrestricted clinical authorization.
- Faculty may receive operational facts such as permitted attendance/status without clinical findings.
- Camp body measurements are restricted to parent/health staff and are excluded from student-facing/comparative responses.
- Health documents are private objects with controlled download authorization.

12.3 School safety

Transport, hostel movement, emergency, health medication and incident workflows are safety-sensitive. For these domains, stale data, duplicate events and state ambiguity must be treated as explicit engineering failure modes rather than ordinary CRUD edge cases.

13. Testing & Acceptance Strategy

13.1 Test layers

Layer	Coverage
Unit	Domain rules, calculations, state transitions, validators
Repository/DB	Constraints, indexes, transaction behavior, RLS and query correctness
Application service	Authorization, orchestration, transaction and idempotency
API integration	HTTP contract, auth, validation, errors, pagination
Concurrency	Wallet races, bed allocation, payment allocation, replay
Device replay	Offline event duplication, stale device state, partial sync
Security	Role/assignment/object isolation, sensitive data, provider callback verification
E2E	Admission, academic, finance, wallet, transport, hostel, health, camp workflows

13.2 Mandatory invariant suite

- Parent -> own child succeeds; unrelated child is hidden.
- Faculty -> assigned class/subject succeeds; unrelated scope fails.
- Locked attendance/marks reject ordinary mutation.
- Payment allocation cannot exceed confirmed payment.
- Refund cannot exceed refundable amount.
- Concurrent wallet debits cannot overspend.
- Duplicate NFC/device local event cannot create duplicate validated event.
- Two concurrent hostel allocations cannot obtain the same bed.
- Health data is inaccessible to unauthorized roles.
- Class transfer/withdrawal does not rewrite historical records.
- Parent failed reset attempt does not consume reset allowance.
- One successful parent self-service reset consumes the allowance; further self-service reset requires Admin.
- Admin-assisted reset is audited.
- Forged payment callback is rejected.
- Replayed payment callback produces no second state change.
- Boarding close marks EXPECTED students NOT_BOARDED and suppresses false alerts for approved leave.
- ALIGHT without BOARD is rejected.
- Standing medication cannot be created without active consent.
- Overlapping active medication schedules return 409.
- Missed medication is recorded rather than deleted.
- Camp cannot complete with unresolved abnormal findings.
- Faculty cannot read restricted camp checkup findings.
- No endpoint accepts a school/campus identifier.
- A Sports Faculty member’s PT-period classroom attendance authority is resolved from their faculty-subject-assignment scope, never from their Sports Faculty (team/sport) assignment scope; holding one never substitutes for the other (Section 1.4, Annex C.2).

13.3 API inventory verification

The CI pipeline should maintain an expected endpoint manifest derived from the corrected API source. The test must compare method + normalized route template and fail on missing, duplicate or unexpected endpoints. The target inventory is 688, including /health and /ready outside /api/v1. expected = loadApprovedApiManifest() actual = introspectNestRoutes()

assert missing(expected, actual) == [] assert unexpected(expected, actual) == [] assert duplicates(actual) == [] assert len(expected) == 688

14. Deployment, Configuration & Operational Boundaries

14.1 Single-school deployment

One deployment serves one configured school product boundary. There is no tenant-routing middleware, school-selection header, campus selector or platform control plane.

14.2 Secrets

- Supabase service-role credentials are server-side only.
- Provider signing secrets are stored in secret management, never database rows exposed to clients.
- Device credentials are scoped and revocable.
- Encryption keys and NFC personalization key material are never returned by API responses.

14.3 Database migrations

1. Create migration.
2. Review constraints/indexes/foreign keys.
3. Apply to development.
4. Run repository/application tests.
5. Run RLS/security tests.
6. Run data integrity checks.
7. Apply to staging.
8. Run E2E/concurrency/replay tests.
9. Apply to production under controlled deployment.
10. Regenerate schema.prisma from the resulting database if that is the project’s chosen generation process.

14.4 Health endpoints

GET /health is a liveness check with no business data. GET /ready verifies readiness without exposing sensitive dependency details. Both remain outside /api/v1 as defined by the corrected API source.

15. Implementation Sequence

1. Freeze the corrected 688-endpoint API inventory, approved schema and this LLD.
2. Create/verify PostgreSQL migrations as physical source of truth.
3. Seed roles, permissions, controlled reference values and stable system configuration.
4. Integrate Supabase Auth and USER_ACCOUNTS mapping.
5. Implement authorization context, role/assignment resolution and RLS defense-in-depth.
6. Implement shared validation, errors, request IDs and structured logging.
7. Implement idempotency, transaction manager and optimistic concurrency infrastructure.
8. Implement identity -> admission -> students -> guardians -> enrolment.
9. Implement academic configuration -> classes -> teaching assignments -> timetable.

10. Implement attendance -> LMS -> assessment.
11. Implement student development -> community -> sports.
12. Implement finance -> payments -> wallet -> canteen.
13. Implement transport -> NFC -> devices -> GPS -> offline synchronization.
14. Implement hostel -> health -> standing medication.
15. Implement incidents -> communication -> documents/evidence -> emergency.
16. Implement approvals -> bulk -> audit -> reporting/scheduled operations.
17. Implement Phase 11 Camps.
18. Run full concurrency, replay, authorization and historical-correctness tests.
19. Integrate frontend/device clients only against frozen contracts.
20. Run production readiness and disaster-recovery verification.

15.1 Dependency rationale

Identity and authorization are first because every downstream domain depends on object/scope access. Finance/wallet/transport/hostel/health are deliberately implemented after shared transaction and idempotency infrastructure because correctness depends on these primitives. Camps are placed after health, communication and student identity because they cross those boundaries.

16. LLD Freeze Gate

- All 688 approved endpoints have a controller/application-service ownership boundary.
- All critical workflows have an explicit transaction boundary.
- All protected operations have server-side authorization.
- All scarce-resource operations have concurrency protection.
- All retry-prone operations have idempotency.
- All consequential historical corrections preserve traceability.
- Sensitive data has explicit authorization and field minimization.
- Parent access is relationship-driven.
- Parent self-service reset is limited to one successful reset.
- No plaintext credentials are stored in product tables.
- Payment provider callbacks are cryptographically verified.
- Wallet/canteen authority is server-side.
- NFC is an identifier mechanism, not a wallet.
- GPS telemetry cannot fabricate attendance/boarding.
- Boarding close and NOT_BOARDED safety behavior are implemented.
- Standing medication requires consent and produces due-dose state.
- Camp completion is blocked by unresolved abnormal findings.
- Audit/outbox are server-generated and transactionally coupled where required.
- Documents use private storage.
- No multi-school/tenant/campus control plane exists.
- No unjustified microservice/Kafka/Kubernetes/distributed infrastructure is introduced.
- API, schema, RLS, LLD and tests are reviewed together.

17. Complete API Baseline Reconciliation

Phase	Feature	Endpoints	Added in v4.0
1	Authentication, Identity & Session	16	—
2	School Academic Configuration & People	31	—
2	User Accounts, Roles & Access	26	—
2	Students, Guardians & Enrolment	33	—
2	Admissions	9	—
3	Classes, Teaching Assignments & Timetable	28	—
3	Attendance	14	5
3	Homework, Assignments, LMS & Syllabus	27	—
3	Assessment, Examination & Report Cards	38	—
4	Student Development & Activities	45	—
4	Community	13	—
4	Sports (Admin: structure/equipment · Sports Faculty: operations)	61	—
5	Finance, Fees & Payments	43	3
5	Wallet & Canteen	39	11
6	Transport, NFC, GPS & Devices	67	10
7	Hostel	67	—
8	Health / Infirmary	43	4
9	Incidents, Discipline, Safety & Emergency	30	—
9	Communication, Notifications, Documents & Evidence	10	1
10	Approvals	5	—
10	Bulk Operations	8	—
10	Audit	3	—
10	Reporting & Scheduled Operations	6	—
11	Camps	26	26
TOTAL		**688**	**60**

(Attendance corrected from 10/1 to 14/5 and TOTAL from 684/56 to 688/60 in this reconciliation — see Section 1.3. School EOS API Documentation v4.0 carries its own note that this stated total and its literal row-by-row count have never fully reconciled by a handful, pre-dating this LLD's corrections; that note is preserved rather than silently resolved.)

This table is the corrected feature inventory from the attached API source.

17.1 Exact 10 Transport/NFC additions

POST /api/v1/transport/trips/{tripId}/boardings

GET /api/v1/transport/trips/{tripId}/boardings

GET /api/v1/transport/trips/{tripId}/student-status

GET /api/v1/transport/trips/{tripId}/exceptions

POST /api/v1/transport/trips/{tripId}/boarding/close

GET /api/v1/students/{studentId}/transport/status

GET /api/v1/students/{studentId}/transport/live

POST /api/v1/nfc/cards/{id}/provisioning-token

POST /api/v1/nfc/cards/{id}/provisioning/confirm

GET /api/v1/nfc/cards/blocklist

These are reproduced exactly from the API correction reconciliation.

17.2 Exact Phase 11 Camps inventory

GET /api/v1/camps POST /api/v1/camps

GET /api/v1/camps/{id}

PATCH /api/v1/camps/{id}

POST /api/v1/camps/{id}/announce

POST /api/v1/camps/{id}/start

POST /api/v1/camps/{id}/complete

POST /api/v1/camps/{id}/cancel

GET /api/v1/camps/{id}/sessions

POST /api/v1/camps/{id}/sessions

GET /api/v1/camps/{id}/services

POST /api/v1/camps/{id}/services

GET /api/v1/camps/{id}/participants

POST /api/v1/camps/{id}/participants

POST /api/v1/camps/{id}/consents

POST /api/v1/camp-participants/{id}/attendance

GET /api/v1/camp-participants/{id}/checkups

POST /api/v1/camp-participants/{id}/checkups

GET /api/v1/camp-checkups/{id}

PATCH /api/v1/camp-checkups/{id}

GET /api/v1/camps/{id}/follow-ups

POST /api/v1/camp-checkups/{id}/follow-ups

POST /api/v1/camp-follow-ups/{id}/complete

GET /api/v1/camp-partners

POST /api/v1/camp-partners

GET /api/v1/students/{studentId}/camps

The 26 Camps endpoints are the new Phase 11 domain.

18. External Engineering Research Basis

18.1 Microsoft DDD / bounded contexts

Microsoft’s DDD guidance describes a cohesive domain model for each bounded context and notes that entities should carry behavior where domain complexity warrants it. The School EOS LLD applies that selectively: simple configuration resources may remain straightforward application services, while finance, wallet, transport, hostel, health and approval domains use richer domain services and explicit invariants.

18.2 Repository and persistence boundary

Microsoft’s persistence guidance describes repositories as a way to keep persistence concerns outside the domain model. School EOS therefore uses repository interfaces at the module boundary and concrete PostgreSQL/Supabase implementations in infrastructure.

18.3 API security

OWASP identifies broken object-level authorization, broken authentication, broken property-level authorization, unrestricted resource consumption, broken function-level authorization, sensitive business-flow abuse, SSRF, security misconfiguration, improper inventory management and unsafe API consumption as key API risks. The LLD's authorization engine, DTO allowlists, rate limits, explicit command/state machines, provider verification and 688-endpoint inventory test are designed against those risks.

18.4 Supabase Auth

Supabase documents Auth as the service responsible for validating, issuing and refreshing JWTs; the Auth schema is not exposed through the auto-generated API. The LLD therefore keeps authentication in the Supabase Auth boundary and product identity in USER_ACCOUNTS.

18.5 PostgreSQL locking

PostgreSQL documents row-level locking including FOR UPDATE, which blocks competing modifications to the locked rows until transaction end. The LLD uses this deliberately for wallets, beds, payment allocations and other scarce-resource operations, combined with database constraints.

19. Final LLD Declaration

Status: This v3.0 document is the detailed LLD implementation baseline after reconciliation against the corrected 684-endpoint API source.

The major incompleteness in the earlier LLD was not a missing architectural layer; it was that the document was generated against the older 628-endpoint contract. The corrected source adds 56 endpoints, including an entire Phase 11 Camps domain, and changes several critical contracts. This edition incorporates those changes and expands the implementation detail so the LLD is not merely an executive summary.

No product-scope invention has been made around multi-school, campus selection or platform Super Admin. Those concepts are explicitly excluded. The LLD also does not replace the database migration authority or silently invent physical tables.

Implementation principle: if an implementation decision is not represented in the product baseline, schema, API contract or this LLD, it must be recorded as an explicit change rather than silently introduced in code.

Engineering north star: correctness first; then security, safety, reliability, traceability, real-world usability, simplicity and extensibility. ## Technical Annex — Final Detailed Implementation Decisions Review purpose: The existing v3.0 master was rechecked against the corrected 684-endpoint API document, its API correction addendum, and the canonical database architecture. This annex expands implementation mechanics instead of replacing approved source material.

Authority: exact API method/path/purpose/access remains the corrected API v2.0 source; PostgreSQL migrations remain physical-schema authority. The annex is the implementation design connecting those authorities.

A. Complete implementation meaning

- Every client-facing endpoint maps to a controller action and an application use case.
- Controllers do not contain business rules or direct SQL.
- Every protected request resolves authentication, product account, effective roles, assignments, object relationship and current state.
- Every consequential mutation defines transaction, idempotency, audit and outbox behavior.
- Every historical correction preserves the original consequential fact.
- Every scarce resource has both application concurrency control and database integrity protection.
- Every external callback defines authentication, replay protection, ordering and unknown-reference behavior.
- Every offline protocol defines durable local event identity and server reconciliation.
- Every sensitive domain defines role-level and object/purpose-level access.
- Every API inventory change is reflected in tests and change control.

A.1 Canonical NestJS structure

```
apps/api/src/
  main.ts
  app.module.ts
  common/
    auth/ authorization/ validation/ errors/
```

```
    idempotency/ transactions/ audit/ outbox/
    storage/ observability/ pagination/
  modules/
    identity/ access/ admissions/ students/
    academics/ attendance/ lms/ assessment/
    development/ community/ sports/
    finance/ wallet/ canteen/
    transport/ nfc/ devices/ hostel/ health/
    camps/ incidents/ communication/
    documents/ approvals/ bulk/ audit/ reporting/
  infrastructure/
    postgres/repositories/
    providers/payment/
    providers/notifications/
    providers/gps/
    jobs/
```

A.2 Dependency direction

```
presentation -> application -> domain
                      ^
                      |
                domain ports/interfaces
                      ^
                      |
                 infrastructure
```

Infrastructure implements ports. Domain code never imports NestJS HTTP objects, provider SDKs or SQL details.

B. Exact request execution pipeline

```
HTTP
 -> requestId/correlation
 -> rate limiting
 -> authentication
 -> USER_ACCOUNT resolution
 -> authorization context
 -> DTO validation
 -> object resolution
 -> state validation
 -> idempotency claim
 -> transaction
 -> domain invariants
 -> repository writes
 -> audit
 -> outbox
 -> commit
 -> response serialization
```

B.1 DTO rules

DTO	Allowed	Never accepted directly
Create	business creation fields	id, audit fields, server totals, ownership
Patch	explicit mutable fields	immutable identifiers, historical facts
Command	transition-specific fields	arbitrary database columns
Query	allowlisted filters/sorts/pagination	raw SQL fragments
Provider	adapter-normalized payload	provider JSON as internal domain authority
Device	event/evidence fields	client balance/status authority

B.2 Canonical response

```
success=true
data=<resource-or-command-result>
meta.requestId=<request id>

success=false
error.code=<stable code>
error.message=<safe message>
error.details=<safe structured details>
meta.requestId=<request id>
```

B.3 Error semantics

- 400 validation/schema failure.
- 401 missing/invalid user, device or provider authentication.
- 403 permission/scope/object authorization failure when existence may be revealed.
- 404 absent or intentionally hidden object.
- 409 state, uniqueness, concurrency or idempotency conflict.
- 422 semantic business invariant failure.
- 429 rate/resource limit.
- 500 unexpected server error.
- 503 required dependency unavailable.

C. Authorization engine

```
authorize(actor, action, resource):
  authenticate(actor)
  roles = activeRoles(actor)
  permissions = resolveRolePermissions(roles)
  assignments = activeAssignments(actor, now)
  object = resolveObjectRelationship(actor, resource)
  scope = resolveOperationalScope(actor, resource)
  state = currentState(resource)
  return policy.allow(action, permissions, assignments, scope, object, state)
```

C.1 Parent

- Student access is derived from active STUDENT_GUARDIANS.
- parentId supplied by a client never proves ownership.
- List queries are scoped before pagination.
- Health/finance/camp fields are additionally filtered by purpose.
- Unrelated child objects may be returned as 404 to avoid existence disclosure.

C.2 Faculty

- Class Advisor, Academic Coordinator, Health In-Charge, stage In-Charge, Community In-Charge, and **Sports Faculty** are assignments, not login roles. None of them — Academic Coordinator included — resolve to a WEB-tagged route; Faculty's AUTH GUARD never issues a session that passes a web-only platform check, regardless of assignment.
- Teaching/class responsibility is resolved from USER_ASSIGNMENTS and academic assignment tables. Sports Faculty's team/sport scope is resolved from the same USER_ASSIGNMENTS mechanism, just a different assignment type — no separate resolver was built for it.
- Unassigned class/subject objects are denied even when the actor is Faculty. Symmetrically, a Sports Faculty member's token is denied for any team/sport/fixture outside their own assignment scope, even though the caller genuinely holds the Sports Faculty assignment for some other team.
- Sports structure/asset writes (sport & category definitions, facilities, equipment master data, coach assignment) are never granted to any Faculty assignment, Sports Faculty included — those remain Admin-only (see Phase 4 — Sports).
- **Physical Training (PT) period classroom duties are resolved from the ordinary teaching/class responsibility check above (USER_ASSIGNMENTS / academic assignment tables), never from the Sports Faculty assignment.** A Faculty member holding Sports Faculty for a team has no PT-period classroom authority for any section unless they ALSO hold an ordinary faculty-subject-assignment for Physical Training against that section — the two checks are independent and neither implies the other (see Section 1.4).
- Sensitive health information requires separate health policy.

C.3 Separation of duties

Approval authorization receives requester and current approver. If the workflow requires separation of duties, requester == approver is denied even if the approver has the permission.

D. Transaction and repository mechanics

D.1 Repository contract

- Repositories expose use-case-specific operations, not unrestricted generic CRUD.
- Transaction-scoped lock methods cannot execute outside a transaction.
- List queries have deterministic ordering and bounded pagination.
- Sensitive queries select only required fields.
- Cross-domain repositories are not called directly by other modules.

D.2 Wallet lock recipe

```
BEGIN
  wallet = SELECT ... FOR UPDATE
  balance = authoritative ledger/balance under same lock
  validate ACTIVE + limits + sufficient funds
  INSERT wallet ledger debit
  INSERT coupled sale if applicable
```

```
  INSERT audit
  INSERT outbox
COMMIT
```

D.3 Hostel bed recipe

```
BEGIN
  bed = SELECT ... FOR UPDATE
  allocation = SELECT active allocation
  if allocation exists: CONFLICT
  validate student has no conflicting allocation
  INSERT allocation
  INSERT audit/outbox
COMMIT
```

D.4 Payment allocation recipe

```
BEGIN
  lock payment
  assert CONFIRMED
  lock obligations in deterministic ID order
  remaining = confirmed - already allocated
  if requested > remaining: reject
  insert allocations
  update derived obligation state
  audit/outbox
COMMIT
```

PostgreSQL row-level locking is used for these scarce/concurrent resources; database constraints remain the final integrity boundary.

E. Idempotency protocol

```
scope = sourceType + sourceId + method + routeTemplate + key
fingerprint = SHA256(canonical(request))

existing?
  completed + same fingerprint -> replay stored outcome
  completed + different fingerprint -> 409
  in progress -> bounded wait/retry policy
new?
  claim key
  execute transaction
  persist final outcome
```

- Financial writes use Idempotency-Key.
- Device writes use client/device event IDs.
- Provider callbacks use provider event ID.
- Duplicate events produce no second business effect.
- Keys are not global across unrelated actors/devices.
- Sensitive provider payloads are not stored merely for replay.

F. Audit and outbox

F.1 Atomic business transaction

```
BEGIN
  authoritative business mutation
  audit.write(serverGeneratedFact)
  outbox.enqueue(requiredSideEffect)
COMMIT

AFTER COMMIT
  worker claims outbox
  provider/notification/integration executes
  result/attempt recorded
  retry with bounded backoff
```

F.2 Audit fields

```
  Field                              Implementation rule

  eventId                            immutable server ID

  occurredAt                         server timestamp

  actorId                            user/device/system identity

  action                             canonical business action

  resourceType/resourceId            target

  requestId                          correlation

  source                             WEB/MOBILE/DEVICE/PROVIDER/SYSTEM

  reason                             required for corrections/overrides

  metadata                           structured, redacted, non-secret
```

Application clients cannot create arbitrary authoritative audit history or update/delete existing audit events.

G. Financial and payment implementation

G.1 Payment lifecycle

```
INITIATED -> PENDING -> CONFIRMED -> RECONCILED
                 |
                 +-> FAILED
                 +-> CANCELLED
```

- Confirmed payment cannot move backward to pending.
- Allocation is separate from provider confirmation.
- Allocation total can never exceed confirmed payment.
- Refund total can never exceed refundable amount.
- Refund is compensating history, not deletion.

G.2 Provider callback

```
raw HTTP bytes
 -> HMAC/signature verification
 -> parse provider payload
 -> claim provider event ID
 -> adapter maps provider state
 -> forward-only transition
 -> commit
 -> 200 acknowledgement
```

- No normal Bearer token is used for the provider callback.
- Invalid signature is rejected.
- Replay of a known event has no second effect.
- Unknown payment reference is acknowledged and queued for reconciliation.
- Provider JSON cannot directly update arbitrary database columns.

H. Wallet and canteen

H.1 Core invariant

- Wallet is owned by the student.
- NFC card is only a credential/identifier.
- Wallet debit is atomic.
- Frozen/inactive wallet fails closed.
- Limits are checked in the same transaction as the debit.
- Client-provided balance or total is never authoritative.

H.2 Canteen sale choreography

```
resolve terminal
 -> resolve card
 -> resolve student
 -> resolve wallet
 -> resolve server product price
 -> calculate server total
 -> lock wallet
 -> validate limits/balance
 -> insert sale
 -> insert debit
 -> audit/outbox
 -> COMMIT
```

H.3 Pre-order

```
create: validate date/student/menu -> calculate -> debit -> create order
collect: validate date/status -> terminal/card -> mark collected idempotently
cancel: validate uncollected -> compensating refund -> audit
```

H.4 Auto-top-up

- One active instruction per wallet.

- Top-up amount must exceed trigger threshold.
- Provider mandate is external and is not itself the wallet balance.
- Canceling the School EOS instruction does not silently claim to revoke the bank mandate.
- Successful top-up is a normal authoritative wallet credit with source metadata.

I. Transport, NFC, GPS and devices

I.1 Boarding state

```
EXPECTED -> BOARDED -> ALIGHTED
EXPECTED -> NOT_BOARDED at close

approved leave/not-travelling
 -> suppressed/ABSENT according to approved mapping
```

I.2 Tap pipeline

- Authenticate device.
- Validate device active/assigned.
- Resolve card and blocklist state.
- Resolve student.
- Resolve active transport assignment.
- Validate trip/vehicle context.
- Check device/local event idempotency.
- Apply BOARD/ALIGHT state rule.
- Append authoritative boarding.
- Audit/outbox in same transaction.
- Return deterministic device result.

I.3 Boarding close

- Lock trip.
- Resolve expected student snapshot.
- Load authoritative boardings.
- Apply approved leave/not-travelling suppression.
- Convert remaining EXPECTED to NOT_BOARDED.
- Queue parent safety alert.
- Queue class-advisor notification for pickup case where specified.
- Repeated close is idempotent.

I.4 GPS

GPS telemetry is evidence/operational data. A GPS point cannot independently create attendance or boarding.

J. Offline synchronization

```
LOCAL
  clientEventId
  deviceId
  localCreatedAt
  payload
  localSequence
       |
       v
SERVER
  authenticate
  validate
  deduplicate
  process
  ACK per item
       |
       +-> ACCEPTED
       +-> DUPLICATE
```

```
      +-> REJECTED
      +-> CONFLICT
```

J.1 Fail-open / fail-closed

Operation	Mode	Reason
Payment	FAIL CLOSED	financial authority
Wallet debit	FAIL CLOSED	no overspend
Access	FAIL CLOSED	security
Canteen sale	FAIL CLOSED	wallet authority
Attendance	FAIL OPEN + reconcile	avoid data loss
Transport boarding	FAIL OPEN + reconcile	avoid safety failure from connectivity

Batch sync processes items independently where the source contract requires it; a device must be able to retry unresolved items without duplicating accepted ones.

K. Health and medication

K.1 Standing schedule

```
student
 -> health authorization
 -> active parent consent
 -> lock active schedules(student, medicine)
 -> reject overlapping active schedule (409)
 -> validate dates/frequency/dose
 -> create schedule
 -> create/derive expected doses
 -> audit/outbox
 -> commit
```

K.2 Medication rounds

- Query due doses in the requested window.
- Order by scheduled time and hostel room where relevant.
- Exclude administered doses.
- Expose overdue doses distinctly.
- Raise MEDICATION_MISSED for overdue unadministered doses.
- Administration is append-only.
- Stopping a schedule requires a reason.
- Stopping a schedule never deletes historical administrations.

K.3 Visit-linked vs standing medication

Existing visit-linked medication remains tied to an infirmary visit. The four new standing-schedule/round APIs provide the longitudinal mechanism required for recurring prescriptions, particularly hostellers.

L. Hostel

L.1 Allocation invariant

```
lock bed
 -> check active allocation
 -> check student conflicting allocation
 -> create allocation
 -> audit/outbox
 -> commit
```

- Two concurrent requests cannot both obtain the same bed.
- Database uniqueness/exclusion constraints remain the final boundary.
- Check-in/check-out/transfer are explicit commands.
- Transfer preserves prior history and creates the new movement/allocation fact.
- Hostel attendance is separate from school attendance.

L.2 Gate pass lifecycle

```
REQUESTED
```

```
  -> REVIEWED
  -> APPROVED / REJECTED
  -> GATE-OUT
  -> GATE-IN
  -> CLOSED
```

M. Camps — complete detailed implementation

M.1 Aggregate structure

```
CAMP
  -> CAMP_SESSIONS
  -> CAMP_SERVICES
  -> CAMP_PARTICIPANTS
       -> CONSENT
       -> ATTENDANCE
       -> CAMP_CHECKUPS
            -> CAMP_FOLLOW_UPS
  -> CAMP_PARTNERS
```

M.2 Lifecycle

```
DRAFT -> ANNOUNCED -> ONGOING -> COMPLETED
DRAFT/ANNOUNCED/ONGOING -> CANCELLED
COMPLETED/CANCELLED = terminal
```

M.3 Completion gate

```
lock camp
assert ONGOING
find abnormal checkups lacking required follow-up
if any -> reject completion
freeze checkups
mark COMPLETED
audit/outbox
commit
```

M.4 Privacy matrix

Actor	Attendance	Clinical findings	Aggregate
Parent own child	Yes	Permitted own-child result	Yes
Faculty/organiser	Yes	No unless separately authorized	Operational
Health staff	Yes	Yes within health scope	Yes
Leadership	Scoped	Restricted	Authorized aggregate

M.5 No separate camp authentication

Camp is a cross-cutting domain. It reuses the existing identity, role, assignment, parent relationship and health authorization mechanisms. No CampUser, CampLogin or separate authentication boundary is introduced.

N. Communication, documents and evidence

N.1 Announcement

```
DRAFT -> SCHEDULED -> PUBLISHED -> EXPIRED/ARCHIVED
             |
             +-> CANCELLED
```

N.2 Delivery callback

- Provider authentication/signature.
- Provider event ID replay protection.
- Map provider state to internal delivery state.
- Forward-only delivery state.
- Out-of-order stale events do not regress state.
- Business transaction is never rolled back because a notification later failed.

N.3 Private storage

- Binary content is private object storage.
- DOCUMENTS stores metadata and business links.
- Download requires authorization to linked object.

- Use short-lived controlled access.
- Never expose permanent public URLs.
- Health, finance and evidence documents have stronger purpose policies.

O. Bulk, reporting and scheduled jobs

O.1 Bulk

```
CREATE
 -> ATTACH INPUT
 -> VALIDATE / DRY RUN
 -> REVIEW
 -> EXPLICIT CONFIRM
 -> EXECUTE
 -> COMPLETE / PARTIAL / FAILED
```

- Validation does not commit final business mutations.
- Explicit confirmation is required before irreversible processing.
- Every row gets a deterministic outcome.
- Job-level audit remains even when individual rows fail.
- Cancellation is only available before irreversible execution.

O.2 Reporting

- Authoritative domain tables remain the source.
- Views/materialized read models are read-only authorities.
- Exports use authorization and data minimization.
- Large reports can execute asynchronously and store output as private documents.
- Partitioning/retention is introduced only after measured data volume/access patterns justify it.

P. Complete API implementation discipline

```
          The corrected API source defines 688 endpoints (628 -> 684 -> 688 across successive reconciliations; see Section 1.3). The endpoint source remains the
exact method/path
          authority. The LLD implementation mapping is deterministic by endpoint class.
```

Class	Controller	Service	Tx	Idempotency	Audit
GET list	list	query service	No	No	No
GET item	get	query service	No	No	No
POST create	create	create service	Usually	When duplicate risk	Yes
PATCH	update	update service	Yes where consistency matters	If retry risk	Yes when consequential
Lifecycle POST	command	transition service	Yes	Yes	Yes
Approval POST	approve/reject	approval service	Yes	Yes	Yes
Provider callback	callback	provider event service	Yes	Provider event ID	Yes
Device sync	sync	sync service	Per item where required	Device event ID	Yes
Report job	create	report service	Job tx	Yes	Yes

P.1 Mandatory endpoint review checklist

- Exact method/path matches API v2.0.
- Correct authentication mechanism.
- Authorization policy exists.
- Object scope is enforced.
- DTO allowlist exists.
- State guard exists where required.
- Idempotency behavior is explicit.
- Transaction boundary is explicit.
- Audit/outbox requirement is explicit.

- Canonical response/error mapping exists.
- Unit/integration/authorization tests exist.
- Route appears in 688-route CI manifest.

Q. Security and privacy gate

- JWT verification centralized.
- Inactive account rejected.
- Roles and assignments server-derived.
- Parent scope relationship-derived.
- Health/evidence fields minimized.
- Provider callback signature verified over raw body.
- Service-role credentials never reach browser/mobile.
- Rate limits protect login, callbacks and expensive operations.
- Unknown query parameters rejected where contract requires allowlisting.
- Uploads have size/type controls and private storage.
- Outbound provider URLs are allowlisted.
- Secrets never enter logs.
- Audit history is not ordinary UPDATE/DELETE data.

Q.1 Threat-to-control mapping

Threat	Primary control
BOLA	object relationship + scope policy
Broken function auth	permission + assignment + state
Property over-posting	DTO allowlists
Replay	idempotency/provider/device IDs
Callback spoofing	raw-body signature
Sensitive disclosure	field/purpose policy
Resource abuse	rate limits/pagination
Inventory drift	688-route CI manifest
SSRF	allowlisted outbound integrations

R. Testing — final engineering gate

R.1 Mandatory invariants

- Parent can access own child and cannot access unrelated child.
- Faculty can access assigned scope and cannot access unassigned scope.
- Locked attendance and marks require correction workflow.
- Payment allocation never exceeds confirmed amount.
- Refund never exceeds refundable amount.
- Concurrent wallet debits cannot overspend.
- Duplicate device event creates one authoritative effect.
- Concurrent bed allocations produce at most one winner.
- Unauthorized health data is denied.
- Historical transfers do not rewrite prior records.
- Parent failed password reset does not consume allowance.
- One successful parent self-service reset consumes the allowance.

- Admin-assisted parent reset is audited.
- Forged payment callback is rejected.
- Replayed payment callback produces no second state change.
- ALIGHT without BOARD is rejected.
- Boarding close produces NOT_BOARDED where applicable.
- Approved leave suppresses false safety alerts.
- Standing medication requires active consent.
- Overlapping active schedules return 409.
- Camp completion is blocked by unresolved abnormal findings.
- Faculty cannot read restricted camp clinical findings.
- No multi-school/campus-selection endpoint exists.

R.2 Concurrency suite

- 100 concurrent wallet debits against a limited balance.
- Two simultaneous claims for one bed.
- Two payment allocation requests racing on the same remaining amount.
- Two identical NFC events arriving simultaneously.
- Two boarding-close requests arriving simultaneously.
- Two overlapping medication schedule creations.
- Two camp-completion requests.

S. CI/CD and schema/API drift

```
PR
 -> lint/typecheck
 -> unit
 -> route inventory = 688
 -> migrations
 -> schema drift
 -> RLS tests
 -> integration
 -> concurrency
 -> security
 -> build
 -> staging E2E
 -> approval
 -> production
```

- API route inventory must fail on missing, duplicate or unexpected method/path combinations.
- Schema drift check compares implementation against migration authority.
- RLS tests run against realistic role/object combinations.
- Backup/restore is tested before production readiness.
- Production schema is never manually edited outside migrations.

S.1 Change-control chain

```
Requirement
 -> domain rule
 -> migration/schema
 -> API
 -> authorization
 -> LLD
 -> tests
 -> implementation
 -> documentation
```

T. Final review declaration

Reviewed against: corrected 688-endpoint API v4.0 (School EOS API Documentation v4.0), the sports/faculty role-model specification, and the previously designed LLD foundation. The earlier 628-endpoint limitation has been explicitly superseded.

- Single-school boundary retained.
- No multi-school, tenant or campus-selection surface.
- No platform Super Admin/Correspondent.
- All 60 API additions accounted for (56 from the original v3.0/v4.0-Annex reconciliation, plus the 4 staff biometric/manual-attendance endpoints this v5.0 edition adds to Section 6 — see Section 1.3).
- Phase 11 Camps fully represented.
- Critical payment/NFC/transport/wallet/hostel/health/approval invariants represented.
- Detailed transaction and concurrency recipes represented.
- Offline and provider callback behavior represented.
- Authorization is role + assignment + scope + object + state.
- Audit/outbox atomicity represented.
- Testing and CI inventory gates represented.
- PostgreSQL migration authority preserved.
- No unjustified microservice/Kafka/Kubernetes/distributed infrastructure introduced.

Final status: The master document plus this annex is the reviewed implementation-level design baseline. It is intentionally detailed rather than executive-summary-only. Where the source documents deliberately leave a physical implementation choice open, this document preserves that boundary instead of inventing an unsupported schema or product feature.